ShadowLedger: A Storage-Scarce Blockchain with Erasure-Coded Fragmented History and Storage-Weighted Leader Election

Anonymous Author(s)

Abstract

Full nodes in conventional blockchains must store the *entire* chain history, an unbounded, monotonically growing cost paid by every node, which centralizes validation around storage-rich operators and wastes resources through total replication. Proof-of-Work (PoW) chains compound this with energy spent purely to order blocks. We present **ShadowLedger**, a permissionless blockchain in which *storage*, not computation, is the scarce, security-relevant resource. Every block body is encoded with Reed–Solomon into $K+M$ shards and distributed by rendezvous (highest-random-weight) hashing, so that *no node stores the full history*: any K-of-$(K+M)$ shards reconstruct a block while fewer than K reveal nothing. The right to produce blocks comes not from hashing but from a Proof-of-Storage discipline over an on-chain, bonded validator registry, gated against Sybil attacks by a one-shot registration PoW and protected by equivocation slashing. Leader election is *weighted by proven storage*: validators submit on-chain storage-proof records for protocol-assigned shards, and a validator's block-production odds scale with its accepted proofs, decaying to a base weight if it stops proving. We are explicit about what remains: *bond* slashing for missed proofs (the prototype penalizes by weight decay, not bond burn) and storage-weighted *fork choice* (branch weight is still chain length) are documented gaps, not hidden ones. A heaviest-chain fork-choice and reorganization engine (state rewind plus replay) with depth-based finality provides convergence and hard irreversibility, and a PoW faucet anchored to committed block data lets capital-less newcomers join. We give the design, an analytical model showing per-node history storage scales as $O(1/n)$ in the number of nodes n, and measurements from a running Go prototype deployed on a public cloud node: at n=128 each node stores $\approx 28\times$ less than a full replica; Reed–Solomon coding sustains 0.3–1.4 GB/s with a fixed 1.5× overhead; seven forged-block attack variants are rejected; two partitioned validators reconverge; and a zero-balance wallet earns its first coins on the live network. We articulate the central *state-versus-history* separation that reconciles "no node stores everything" with stateless validation, present a threat model, and state the open problems (storage-weighted fork choice, succinct state commitments, external audit) candidly.

CCS Concepts

• **Security and privacy** → **Distributed systems security**; • **Theory of computation** → *Distributed algorithms*; • **Information systems** → *Data management systems.*

Keywords

blockchain, erasure coding, Reed–Solomon, rendezvous hashing, proof-of-storage, data availability, consensus, finality, smart contracts

1 Introduction

A defining and growing cost of operating a full node in Bitcoin [24] or Ethereum is storing the complete transaction history. This cost is *unbounded*, it only ever increases, and it is paid by *every* full node, because the classical security model replicates the whole chain everywhere. The practical consequence is centralization: as history accumulates, fewer participants can afford to validate independently, and the network drifts toward a small set of storage-rich operators. Energy is the second well-known cost: PoW chains expend electricity solely to decide who appends the next block, work that is intrinsically discarded.

The magnitude is not hypothetical. Production chains accumulate history into the hundreds of gigabytes and beyond, and the requirement that every validating node retain all of it is widely recognized as a scalability and decentralization bottleneck, motivating an entire line of coded-storage work [26, 32]. Yet most such work treats coding as an optimization bolted onto an unchanged consensus. We instead ask what a chain looks like when fragmentation is the *default* and when block-production rights are derived from the very act of storing fragments.

We take the position that the scarce, security-relevant resource of a blockchain should be *storage of the network's own data*, not wasted hashing, and that *no single node should need the entire history* to participate. **ShadowLedger** is our realization of this position. Its thesis has two parts. First, block *history* is erasure-coded and scattered across the network so each node retains only a fraction, yet any sufficiently large set of nodes can reconstruct any block on demand. Second, the right to produce blocks is earned by *proving storage* of assigned fragments over a bonded validator set, rather than by burning energy; the only PoW present is a one-shot Sybil gate and a small onboarding faucet, never the block-ordering mechanism.

Realizing this is not free of tension. If history is not stored whole, validators must still be able to recover it, or a malicious producer could withhold data; this is the *data-availability* problem [2, 33]. If block production is permissionless, the system must resist Sybil flooding and equivocation. And if storage replaces hashing, the protocol must measure and reward it without a centralized auditor. We address each of these and report what a working implementation achieves and what it does not.

Contributions.

(1) An architecture that fragments *history* (block bodies) with Reed–Solomon coding and rendezvous placement while keeping *state* (balances, nonces, contract storage) materialized locally, the *state-versus-history* separation that reconciles "no node stores everything" with fast stateless validation (§5).
(2) A Proof-of-Storage consensus with a bonded on-chain validator registry, a one-shot registration PoW as a Sybil gate, HRW leader election with a round-based liveness fallback, equivocation slashing, a heaviest-chain reorg engine, and

depth-based finality (§6, §8). Leader election is weighted by on-chain proven storage (validators submit storage-proof records that drive election odds); what remains specified-not-built is *bond* slashing for missed proofs (the prototype uses weight decay) and storage-weighted *fork choice* (§7, §14).

(3) A capital-free onboarding mechanism (a PoW faucet) and a deterministic contract VM with a Solidity-like language, both anchored to committed block data and thus verifiable without holding the fragmented body (§9, §10).

(4) An analytical storage model ($O(1/n)$ per-node history) and an empirical evaluation of a live Go prototype, with an explicit threat model and an honest account of open problems (§11–§14).

The paper is organized as follows. §2 and §3 review the primitives (erasure coding, rendezvous hashing, data availability, storage-based consensus) and position ShadowLedger against prior coded blockchains, sharding, and proof-of-storage systems. §4 states the design goals and gives the node architecture. §5 develops the erasure-coded history and its storage model; §6–8 the Proof-of-Storage consensus, storage auditing, fork choice, and finality; §9 the economics and capital-free onboarding; and §10 the contract VM. §11 presents the threat model, §12 the implementation, and §13 the evaluation. §14–15 discuss limitations, applicability, and design rationale before §16 concludes.

2 Background

Reed–Solomon erasure codes. A Reed–Solomon (RS) code [27] encodes K data symbols into $K{+}M$ symbols such that the original is recoverable from *any* K of them; it tolerates up to M erasures. Compared with r-fold replication (which costs $r\times$ storage to survive $r{-}1$ losses), an (K, M) RS code survives M losses at only $(K{+}M)/K$ overhead. Crucially, RS is all-or-nothing below the threshold: fewer than K symbols determine the message no better than zero symbols do, which doubles as a confidentiality property for partial holders.

Worked example. Take $K{=}4, M{=}2$, so $T{=}6$ shards tolerate $M{=}2$ erasures at $T/K = 1.5\times$ overhead. A 1 MiB body splits into four 256 KiB data shards plus two 256 KiB parity shards; any four of the six rebuild the megabyte. Triple replication would instead store 3 MiB to survive two losses, twice the cost for the same fault tolerance. This 1.5×-versus-3× gap, generalized over a growing node set, is the lever ShadowLedger pulls (§5.3).

Rendezvous (HRW) hashing. Given a key and a set of nodes, highest-random-weight hashing [29] ranks nodes by $h(\text{key}, \text{node})$ and assigns the key to the top-ranked node(s). Unlike ring-based consistent hashing [16], each key's ranking is independent, so adding or removing a node reshuffles only the keys that node would win or lose, ideal for shard placement under churn. Formally, for a key κ, node set V, and replication r, the holders are the top-r nodes by weight,

$$\textsc{Holders}(\kappa, V, r) \;=\; \text{top-}r_{v\in V}\; H(\kappa \,\|\, v),$$

and a single highest-weight node is the special case $r{=}1$ used for leader election with $\kappa = (\mathit{prevHash}, h, t)$. Adding a node v' changes the holder set of κ only if $H(\kappa\|v')$ ranks within the top r, so the expected fraction of keys reassigned on a single join is $r/|V|$, which is the minimal disruption a placement scheme can achieve. We use HRW for both shard placement and leader election, keeping one source of deterministic, agreement-free randomness.

Data availability. When blocks are erasure-coded and dispersed, light or partial nodes need assurance the data *exists* and can be reconstructed. Fraud and data-availability proofs [2], coded Merkle trees [33], and the formal theory of data-availability sampling [11, 12] show that with 2D/coded commitments a verifier can probabilistically check availability by sampling a few shards: if a fraction of the coded data is missing, an erasure-coded block cannot be reconstructed, and that missing fraction is exactly what random sampling is overwhelmingly likely to hit, so a withholding producer is caught with few queries. These results target light clients that never store blocks. ShadowLedger's setting is different, its participants *do* store shards, so it uses a simpler operational notion suited to a fragment-holding network (§5): a block is canonical only if the network can reconstruct it to its signed commitment, and a node that needs a body gathers K shards and verifies the hash. Layering succinct DA sampling on top, to protect non-storing light clients, is compatible and left as future work.

Storage-based consensus. Replacing PoW's wasted work with a useful resource is a recurring goal. Proofs of space [8] and space-time [23] prove dedicated disk; SpaceMint [25] and Chia [7] build currencies on them. Proofs of retrievability [14], proof-of-replication [9], decentralized storage networks such as Storj [31], and Permacoin [21] bind rewards to stored data. The energy contrast is the motivation: PoW chains spend electricity per block forever, whereas storing data is a one-time cost amortized over the data's lifetime, so a storage-based right to produce blocks converts an ongoing burn into a useful, reusable asset. ShadowLedger's Proof-of-Storage is closest to Permacoin in spirit but applies to the chain's *own* erasure-coded history rather than an external archive, so the resource being proved is the very data the network must keep anyway.

3 Related Work

Coded and low-storage blockchains. Several systems erasure-code blocks to shrink per-node storage: low-storage nodes that keep RS fragments [26], secure fountain (LT-code) architectures [15], entropy-based downsampling [13], and failure-pattern-aware codes [22]; a recent survey maps the field [32]. These works typically retain a conventional consensus and treat coding as a storage optimization layered on top. ShadowLedger instead makes fragmentation the *default* for all history *and* couples shard placement to the consensus membership set, so storage and block-production rights are one mechanism rather than two. The closest points of comparison differ in emphasis: erasure-coded low-storage nodes [26] keep RS fragments but rely on a fixed set of full nodes to bootstrap from, and the secure-fountain architecture [15] analyzes precisely the storage-versus-bootstrap-bandwidth tradeoff we revisit empirically in §13; coded Merkle trees [33] and downsampling [13] optimize availability proofs and per-node footprint respectively. None of these ties the act of *storing a shard* to the right to *produce a block*, which is ShadowLedger's organizing idea.

Data-availability designs. LazyLedger/Celestia [1] reduces block validity to data availability; coded Merkle trees [33] and

Table 1: Positioning of ShadowLedger against representative prior work.

System	History storage	Block production	Placement
Bitcoin [24]	full replica	PoW	n/a
Coded nodes [26]	RS fragments	inherited	ad hoc
SpaceMint [25]	full replica	PoSpace	n/a
Permacoin [21]	external data	PoRet	assigned
OmniLedger [18]	sharded state	PoS/BFT	committee
ShadowLedger	RS fragments	Proof-of-Storage	HRW

DA sampling [11, 12] give succinct availability guarantees. ShadowLedger shares the availability-first philosophy but targets a fragment-storing full-node network rather than a light-client sampling regime; succinct DA proofs are complementary future work (§14).

Sharding. Elastico [19], OmniLedger [18], RapidChain [34], and Monoxide [30] scale throughput by partitioning *transactions/state* across committees. ShadowLedger instead partitions *storage of history* while keeping a single global state and a single ordering, so it does not inherit cross-shard atomicity costs; the lineage's use of one-shot PoW for committee randomness directly informs our registration gate.

Storage markets versus storage ledgers. Filecoin [9] and Storj [31] are decentralized storage *markets*: their purpose is to store arbitrary client data for pay, with a blockchain as the accounting layer beside that storage. ShadowLedger inverts the relationship. The data being stored *is* the chain's own history, and storing it *is* the consensus right, so there is no separate payload market and no oracle problem of proving that externally supplied data was kept; the verifier already holds the commitment in a signed header. The cost of this simplicity is generality: the only thing the network stores is itself. Permacoin [21] sits between the two, repurposing mining to preserve a fixed external dataset, which ShadowLedger removes entirely.

Consensus and finality. Our heaviest-chain rule descends from GHOST [28]; depth-based finality is grounded in the common-prefix guarantees of the Bitcoin backbone analysis [10]. The bonded-validator layer follows Proof-of-Stake practice, Ouroboros [17] for the security model, Tendermint [3] and Casper/Gasper [5, 6] for deposits, equivocation slashing, and finality gadgets. Light-client sync over fragmented history relates to FlyClient [4] and Merkle commitments [20].

4 System Overview

A ShadowLedger node runs five cooperating subsystems (Figure 1): (i) an *erasure layer* that encodes and reconstructs block bodies; (ii) a *placement* module computing HRW shard assignments from the live membership; (iii) a *consensus engine* (Proof-of-Storage leader election over the bonded registry); (iv) a *chain manager* holding the fork-choice tree, reorg, and finality logic plus the local state; and (v) a *peer-to-peer* layer with DNS-seed and peer-exchange discovery (no central server). The lifecycle of a block is: a leader assembles transactions, computes the state transition, signs a header committing the body hash and per-shard hashes, and broadcasts the block; each node applies the state transition, then keeps only its HRW-assigned shards and discards the full body. Reconstruction is on demand: a node that needs an old body gathers K shards from current holders and checks them against the committed hashes.

Table 2: Notation.

symbol	meaning
n	number of live nodes
K, M	RS data / parity shards; $T = K+M$
r	holders per shard (replication factor)
$\|b\|$	block body size
f	per-node unavailability probability
F	finality depth (blocks behind head)
N	faucet PoW difficulty (leading zero bits)
τ	target block time

Design goals. The architecture is shaped by five requirements that the rest of the paper returns to. **G1 (storage scarcity):** per-node history cost should shrink as the network grows, not stay constant (§5.3). **G2 (fast validation):** validating a new block must not require a network round-trip, so the working set stays local (§5). **G3 (permissionless, Sybil-resistant entry):** anyone may join as a validator, but identities must carry a cost (§6). **G4 (convergence and irreversibility):** temporary partitions must heal, and sufficiently deep history must become unchangeable (§8). **G5 (capital-free onboarding):** a participant with no tokens must have a path to acquire a small amount without a trusted faucet operator (§9). No single mechanism meets all five; the contribution is their composition.

5 Erasure-Coded History

5.1 State versus history

The pivotal design decision is to treat *state* and *history* differently. *State*, account balances, nonces, contract storage, is the materialized result of replaying history; it is bounded (one entry per active account or storage slot) and every validator keeps it in full locally, exactly as Bitcoin keeps the UTXO set and Ethereum the state trie. *History*, the block bodies that produced that state, is unbounded and is the *only* thing fragmented away. Because state is derivable, losing a node's local copy is not data loss: it is recomputed by replay, or fetched as a snapshot from a peer and checked against the committed chain. This separation is precisely what lets "no node stores all history" coexist with fast, deterministic validation: validation reads *state*, which is local; only audit, sync, and dispute resolution need *history*, which is recoverable.

5.2 Encoding, placement, reconstruction

On commit, a block body b of size $|b|$ is RS-encoded into $T = K+M$ shards, each of size $\lceil |b|/K \rceil$ (data shards padded; parity shards equal size). The erasure shape (K, M) is chosen adaptively from the live node count n (§4); the leader stamps it into the signed header, so peers read the shape rather than agreeing on it. Each shard index s is assigned to the top-r nodes under HRW($blockID, s$), where r is a replication factor that also scales with n. A node persists only the shards it wins and discards b. The header commits $BodyHash(b)$ and the vector of per-shard hashes; any node gathering K valid

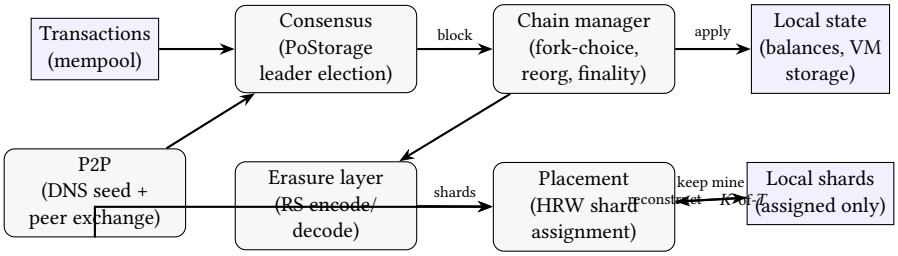

Figure 1: ShadowLedger node architecture. A leader assembles transactions, the chain manager applies the state transition (kept local) and hands the body to the erasure layer; HRW placement keeps only the node's assigned shards and discards the full body. Old bodies are reconstructed on demand from any K-of-T shards gathered over the P2P layer.

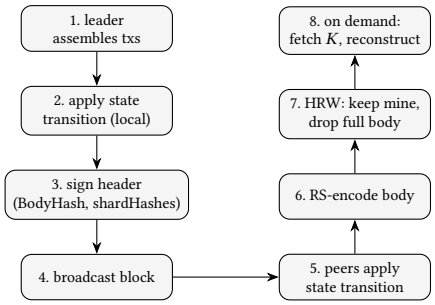

Figure 2: Block lifecycle: state is applied and kept by all; the body is encoded, only assigned shards are retained, and old bodies are reconstructed on demand.

shards reconstructs b and verifies *BodyHash* = commitment, so a lying holder is detected and skipped.

Concrete walkthrough. Consider a 128-node network producing block 1000 with a 1 MiB body. The shape is K16 M8, so the body becomes 24 shards of 64 KiB each, and the header commits one 32-byte hash per shard plus the body hash. Node X computes $\mathrm{HRW}(id_{1000}, s)$ for every shard s and finds it ranks in the top r=3 for, say, shards $\{3, 7\}$; it stores those two (128 KiB) and discards the 1 MiB body. Months later an auditor requests block 1000: it queries holders, collects any 16 of the 24 shards (each verified against its committed hash), runs RS decode to rebuild the 1 MiB body, and checks the body hash. If eight holders are offline the remaining sixteen still suffice; if a malicious holder returns a corrupted shard, its hash mismatch excludes it and another holder's shard is used instead. Node X never reconstructed the block to validate the chain ahead of it; it only did so on this explicit audit request.

5.3 Storage model

Let each shard have r holders and let the T shards be spread over n nodes by HRW. The expected number of shard copies a node holds for one block is Tr/n, each of size $|b|/K$, so expected per-node storage per block is

$$S_{\text{node}} = \frac{Tr}{n} \cdot \frac{|b|}{K}.$$

Full replication stores $|b|$ per node, giving a reduction factor

$$\rho = \frac{|b|}{S_{\text{node}}} = \frac{Kn}{Tr}.$$

With ShadowLedger's policy (T, K saturate at K=16, M=8 and r=3 for large n), $\rho \approx Kn/(Tr) = 16n/(24 \cdot 3) = n/4.5$, i.e. per-node history cost falls as $O(1/n)$ while full replication is constant. Below $n \approx 4$ the fixed parity overhead T/K dominates and fragmentation is not yet a win, an honest small-network regime we report rather than hide. §13 validates this model empirically against rendezvous placement.

5.4 Sync and state snapshots

A node that joins late, or that lost its local state, must reach the current head without necessarily replaying every historical body. Two paths exist. The *header-first* path downloads the (small, signed) header chain and verifies its links and signatures, giving the node a trustworthy skeleton of heights, commitments, and the validator set; bodies are then fetched and reconstructed only as needed (§13 quantifies that cost). The *snapshot* path fetches a materialized state at a recent height from a peer and adopts it as a *checkpoint*: the node sets its replay base (§8) to that height, so it can still reorg above the checkpoint but trusts the snapshot below it. Because the snapshot is the fold of all prior blocks, its correctness is in principle checkable by replay; making it succinctly *verifiable* without replay needs a committed state root, which the present header omits (§14). In both paths, steady-state operation then reverts to keeping only HRW-assigned shards: sync cost is paid once, storage savings persist.

6 Proof-of-Storage Consensus

Bonded validator registry. Validators are on-chain state, not configuration. A node joins by submitting a *register* transaction that locks a bond (skin-in-the-game, in the spirit of PoS deposits [5, 17]) and solves a one-shot registration PoW. The PoW is *not* block production: it is a Sybil gate that makes mass fake identities expensive and seeds shard-assignment randomness, exactly as one-shot PoW is used for committee formation in Elastico [19]. The active registry is read live from state, so registration changes the producer set with no reconfiguration round and every node derives the same set deterministically.

The two entry costs are complementary. The *bond* is recoverable capital that creates skin-in-the-game: it is returned on a clean exit but burned on equivocation (§8, 11), so it deters misbehavior rather

Algorithm 1: Proof-of-Storage leader election with round fallback (run each tick).

Input: height h, parent header P, local time *now*, block time τ, my id *me*
$V \leftarrow$ ActiveValidators() `// live from on-chain registry`
$t \leftarrow \max(0, \lfloor (now - P.ts)/\tau \rfloor)$ `// elapsed rounds`
$leader \leftarrow \arg\max_{v \in V}$ HRW$(P.hash, h, t, v)$
if *leader* = *me* ***and*** *have-work* **then**
  build block at (h, t) with $ts \geq P.ts + t\tau$; sign; broadcast
end

than entry. The *registration PoW* is a one-time sunk compute cost that deters *mass* entry: minting m Sybil identities costs m independent puzzle solutions, making large fake fleets expensive even if each bond is small. The difficulty is a tunable parameter, set so a single honest registration is sub-second while a fleet of thousands is costly; §13 reports the analogous cost curve for the faucet PoW, which uses the same primitive. Because the puzzle is bound to the registering address, its solution doubles as an unbiased seed for that identity's shard-assignment and leader-election ranking, so a validator cannot choose which shards or slots it will win.

Storage-weighted leader election. For height h and round t, the leader is chosen by HRW over the active validators seeded by $(prevHash, h, t)$, but *weighted by proven storage*: each validator is given $1 + \min(score, C)$ virtual HRW entries, where *score* is its count of accepted on-chain storage proofs (§7) and C caps the bonus. The weight is integer-only, so every node computes the identical election from deterministic chain state, and a validator that stops proving decays to the base weight of one. The rank depends on the previous block hash, so it is unpredictable until the parent is fixed and cannot be ground out in advance (§11). Block-production odds therefore track demonstrated storage rather than mere registration.

Implementation status (what is built vs. specified). To avoid overclaiming, we separate the two precisely. *Built and evaluated*: the bonded on-chain registry, the one-shot registration PoW, the on-chain storage-proof transaction and its verification against committed shard hashes (§7), storage-weighted HRW leader election driven by those proofs with round fallback, equivocation slashing, the reorg engine, and depth-based finality. *Specified but not yet built*: slashing a validator's *bond* for missed storage proofs (the prototype applies the softer penalty of election-weight decay, not bond burn) and storage-weighted *fork choice* (branch weight remains chain length, §8). The election is genuinely storage-coupled in the running binary; what is deferred is the harsher economic punishment and the extension of weighting from leader selection to branch selection.

Liveness fallback. A silent leader must not stall the chain. Round t is derived from elapsed time since the parent ($t = \lfloor \Delta/\tau \rfloor$ for block time τ); if round-0's leader produces nothing within one block time, round 1 elects a *different* validator, and so on. Headers carry t, and a block is rejected unless its timestamp is consistent with its claimed round ($ts \geq prevTs + t\tau$), so a high round cannot be claimed early. Genesis uses a fixed timestamp to anchor the schedule identically on all nodes (Algorithm 1).

Slashing. Two validly signed, conflicting headers at the same height by one validator are provable equivocation. A *slash* transaction carries the two headers as evidence; it burns the offender's bond (a fraction rewards the reporter) and permanently bars the identity, following Casper/Tendermint practice [3, 5].

7 Storage Auditing

Proof-of-Storage is only as strong as the network's ability to *check* that a node holds what it claims. ShadowLedger runs a lightweight challenge–response audit, in the lineage of proofs of retrievability [14] and replication [9] but specialized to committed shards.

Challenge–response. Periodically, a node challenges a holder for one of its assigned shards $(blockID, s)$ with a fresh random nonce c. The holder replies with $\sigma = H(shard \,\|\, c)$. Because the header commits the shard's hash, a challenger can detect a holder that does not actually possess the bytes: a holder without the shard cannot produce a σ consistent with the committed shard (it would have to invert H), and a holder that serves a *wrong* shard is caught when any reconstructing node compares the committed hash. The nonce prevents replaying a previously observed response, so the holder must have the shard *now*, not merely once.

Scoreboard. Each node keeps a local scoreboard of pass/miss outcomes per peer. A peer that repeatedly fails challenges, or serves shards inconsistent with their commitment, is down-scored and, for outright forgery, struck from the local peer set. The scoreboard is a fast local hygiene signal; the consensus-binding counterpart is the on-chain storage score described next, which §6 reads so a validator that proves it stores more of its assigned shards wins more rounds.

On-chain storage-proof records. The audit is made consensus-visible by a storage-proof transaction. A validator periodically proves it can produce a *protocol-assigned* shard of a recent confirmed block: the target block lies within a recency window and at least a few blocks deep, and the shard *index* is derived deterministically as $H(addr \,\|\, blockID) \bmod T$, so the validator cannot cherry-pick one easy shard but must answer for the slot the protocol assigns it (it serves the shard locally if it holds it, or reconstructs it from K peers). Every node verifies the submitted bytes against the committed shard hash for that slot, taken from the persisted header and shard set, so verification needs neither the full body nor any new header field. An accepted proof increments the validator's on-chain *storage score* (capped), which §6 reads to weight leader election; the score decays to the base if the validator stops proving within the window.

Status and honest scope. What this buys, in the running binary, is storage-*coupled* election: block-production odds rise with accepted proofs and fall when proofs lapse, a real economic pressure to store and serve assigned data. What it does *not* yet do is burn a validator's *bond* for missed proofs (the penalty is weight decay, softer than a slash) or weight *fork choice* by storage (branch weight is still chain length, §8). The local scoreboard remains as a fast peer-hygiene tool alongside the on-chain record. We separate these precisely (§6, §14) rather than let the term "proof-of-storage" imply more than is enforced.

Algorithm 2: Depth-based finality advance (after the head changes).

Input: head H, finality depth F, replay base (B, *baseHt*)
if $H.height - F \le baseHt$ **then return**
$target \leftarrow H.height - F$
walk $H \rightsquigarrow B$, collecting canonical blocks with height $\le target$
$S \leftarrow$ Clone(*baseState*); replay collected blocks onto S in order
advance replay base to deepest replayed block; persist; **base never moves back**
`// a fork below the base cannot reach it ⇒ reorg refused`

8 Fork Choice, Reorganization, and Finality

Fork choice. Nodes maintain a block tree and follow the heaviest branch, after GHOST [28]; ties break by lowest block id for determinism. Today "weight" is chain length. We deliberately keep this distinct from *leader election*, which *is* storage-weighted (§6). Extending storage weight to branch selection is not a free relabelling: a branch's weight must be computed identically by every node for any competing branch, yet a producer's storage score is *branch-dependent state* (it depends on which proofs that branch accepted). Making fork choice storage-weighted therefore requires either committing each producer's score in the (signed) header or recomputing weights by replaying each branch, and a naive local-state reading would let nodes disagree on the heaviest branch, a consensus-safety hazard. We treat this as future work rather than risk that on a live network; the leader-election weighting, whose inputs are the post-parent state both producer and verifier already share, has no such hazard.

Reorg engine. On observing a heavier branch, a node rewinds state to a *replay base* and replays the winning branch onto a fresh copy, committing only if the whole branch applies cleanly, so a malformed branch can never corrupt live state. A linear extension of the current head is fast-forwarded without a full replay.

Depth-based finality. The replay base advances to $head - F$ as the chain grows and never moves backward. Because a reorg can rewind no further than the replay base, a competing branch that forks *below* the finalized height $head - F$ literally cannot be applied: it cannot reach the base to replay from, and is refused. Thus history older than F blocks is irreversible. The safety of depth-based finality rests on the common-prefix property: for an honest-majority bonded set, the probability that an F-deep block is later displaced decays exponentially in F [10]; the prototype sets F=16 (Algorithm 2).

Safety and liveness, informally. Safety of finality reduces to a single invariant: the replay base is monotone non-decreasing in height. Since every reorg target is reached by walking down to the replay base and a fork below it cannot be walked to, no committed block at height $\le$ *baseHt* is ever rewritten. The base advances only to $head - F$, so the window of mutable history is bounded by F, and the probability that an honest F-deep block is displaced before finalization decays as the common-prefix bound in F [10, 28]. Liveness is provided by the round fallback of §6: if the round-t leader is silent, round t+1 elects a different validator, so as long as some honest validator is eligible and online the head advances and, with it, the finalized height. The two properties are independent: the round mechanism guarantees progress, while the monotone base guarantees that progress, once F deep, is permanent.

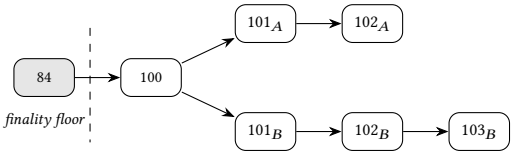

Figure 3: Reorg above the finality floor. Branch B (heavier, height 103) wins over branch A; nodes rewind to the common parent 100 and replay B. Blocks at or below the floor (height 84, shaded) are final and cannot be reorganized.

Worked reorg. Suppose a partition leaves validator A extending tip $\ldots \to 100 \to 101_A \to 102_A$ while validator B extends $\ldots \to 100 \to 101_B \to 102_B \to 103_B$ from the same parent at height 100. When the partition heals, every node sees both branches; B's is heavier (height 103 versus 102), so a node currently on A's tip walks back to the last common point (height 100, which is at or above its replay base), rewinds its state to that point, and replays 101_B, 102_B, 103_B onto a fresh copy. It commits only after all three apply cleanly, then adopts 103_B as head; nodes already on B simply keep it. If instead B's branch had forked at height 80 while the replay base had already advanced to 84 (finality depth reached), the walk-back could not reach a replayable base and the reorg would be refused outright, since heights ≤ 84 are final. Convergence above the finality floor, immutability below it.

9 Economics and Capital-Free Onboarding

Issuance. The native unit $SHARD follows a Bitcoin-style schedule: a fixed maximum supply and a block subsidy that halves every fixed number of blocks, paid to the block's validator as a coinbase together with the transaction fees of that block. Two invariants keep the schedule deterministic and non-inflationary. First, only the subsidy counts against the cap; fees are recycled from payers to the producer, never newly minted, so total issuance is exactly the sum of the (geometrically decaying) subsidies and is bounded by the cap. Second, the faucet pays from a pre-funded treasury balance rather than minting, so onboarding grants do not perturb the schedule. A node validates the coinbase against the schedule on every block, rejecting any block whose coinbase exceeds the height's allowed subsidy plus its fees, so a producer cannot pay itself more than the protocol permits.

PoW faucet. A brand-new follower holds no $SHARD and can pay neither fees nor a bond. The faucet is the on-ramp: the follower mines a nonce so that $H(chainID \,\|\, addr \,\|\, anchorBodyHash \,\|\, nonce)$ has N leading zero bits, and claims a small fixed payout drawn from the treasury (*not* newly minted, so the cap is untouched). The anchor is a recent block's committed body hash, present in the signed header, hence verifiable by *every* node without holding the fragmented body, the same data-availability-safe principle as the rest of the design. The anchor is unknown until the block exists, so claims cannot be precomputed; a per-address cooldown and the

Algorithm 3: Faucet claim verification (Lz: leading-zero-bit count).

Input: claim tx from a at height h: (anchorHt, nonce); params $N, F_f, cool, amt$

if $anchorHt + F_f > h$ **or** $h - anchorHt > 256$ **then** reject
// unconfirmed/stale

$anchor \leftarrow$ HeaderBodyHash($anchorHt$) // from signed header; no body needed

if $Lz(H(chainID \| a \| anchor \| nonce)) < N$ **then** reject
// bad PoW

if $h < lastClaim[a] + cool$ **then** reject // cooldown

treasury $-= amt$; balance$[a] += amt$; $lastClaim[a] \leftarrow h$

tunable hashing cost rate-limit abuse, giving a Sybil price linear in the number of identities (Algorithm 3).

10 Deterministic Smart Contracts

We treat programmability as a *supporting capability*, not a scientific contribution of this paper: the core results concern fragmented history and the storage-coupled consensus, and a reader interested only in those may skip this section. We include it because a storage-scarce ledger is of little use if it cannot run application logic, and because the contract layer must obey the same commitment discipline (signed roots, deterministic execution) as the rest of the system; we therefore keep the description brief.

Contracts execute on a deterministic stack virtual machine with metered gas. Determinism is a hard requirement: every node that runs the same bytecode on the same input and storage must reach the same result, or consensus over contract state breaks. The VM therefore excludes every source of nondeterminism, namely wall-clock time, floating point, and randomness, and bounds execution by a gas budget and a step limit so a contract cannot loop forever or exhaust memory.

Machine model. The VM is a stack machine over 64-bit words with a per-contract key/value store (uint64 to uint64). Opcodes fall into the classes of Table 3. Storage writes are the expensive operation, reflecting their real cost (they enlarge the state every node keeps), so gas pricing discourages gratuitous writes. Cross-contract calls are allowed but bounded by a call-depth limit, which caps reentrancy and recursion. Because a 64-bit word cannot hold a 20-byte address or a string, a byte-storage layer holds variable-length blobs (addresses, token identifiers, URIs) sourced from calldata, with opcodes to store, compare, hash, and return them; this is how a contract can manipulate full addresses despite the uint64 core.

The SHL language. Writing raw bytecode is error-prone, so contracts are written in SHL, a small Solidity-like language compiled through a lexer, parser, and code generator. SHL offers named `state` variables and mappings (each assigned a fixed storage slot by the compiler, so there are no hand-managed magic offsets), functions with selector dispatch, `require` and `revert` with full state rollback, and overflow-checked arithmetic by default. A function selector is derived from the function name and placed in the call's first calldata word; the compiler emits a dispatcher that routes to the matching function and reverts on an unknown selector. Listing 1 shows a fungible-token transfer: the `require` guard reverts the entire call (restoring all storage) if the sender is short, and both updates are overflow-checked, so a short and readable contract is also a safe one.

Table 3: VM opcode classes (representative, not exhaustive).

class	opcodes / role
arithmetic	add/sub/mul (overflow-checked), div/mod (revert on 0)
logic/compare	and, or, not, eq, lt, gt, iszero
stack	push, pop, dup, swap
control flow	jump, jumpi, stop, return, revert
storage	sload, sstore, mix (mapping-slot hash)
environment	caller, value, self, calldata, calldata-len
byte layer	bstore, beq, bhash, caller-bytes, return-bytes
events / calls	log (committed in LogsRoot), call (depth-bounded)

Listing 1: A safe ERC-20-style transfer in SHL.

```
state balances;          // mapping holder -> amount
state totalSupply;

fn transfer(to, amount) {
  require(balances[msg.sender] >= amount);
  balances[msg.sender] = balances[msg.sender] - amount; // checked
  balances[to] = balances[to] + amount;                 // checked
  emit(TRANSFER, msg.sender, to, amount);
}
fn balanceOf(who) { return balances[who]; }
```

Scope. SHL is deliberately a faithful *subset* of the Solidity model rather than a clone. It omits 256-bit integers, contract inheritance, and the ABI, trading generality for a VM small enough to audit and reason about, which suits a system that has not yet undergone external security review (§14). Events emitted via `log` are committed through the block header's logs root, so a verifier can later prove that an event occurred without trusting the node that serves it, the same commitment discipline used for shards and transactions.

11 Threat Model and Security Analysis

We assume an adversary that can run nodes, register as a validator (paying bond + PoW), send arbitrary messages, and partition the network temporarily, but cannot break ed25519 signatures or SHA-256, and does not control a majority of the bonded stake.

Block injection / poisoning. Every header field is signed and validated; each shard is committed by hash. An outsider cannot insert a block: forged signatures, unauthorized producers, tampered bodies (Merkle/body-hash mismatch), wrong height or parent, future timestamps, and forged log roots are all rejected (§13 exercises seven such variants). A lying shard server is detected by the committed shard hash and skipped.

Data withholding. A producer could publish a header but withhold body shards. The availability-first canonical rule treats a block that cannot be reconstructed to its commitment as not-yet-valid; reconstruction succeeds as long as K honest holders exist, which the replication factor r and HRW spread are sized to ensure. Succinct DA sampling [12] would strengthen this from an operational check to a probabilistic proof and is future work.

Table 4: Recoverability at K16 M8, r=3, T=24. Shard loss f^3; block lost iff $> M$ shards lost.

f	shard-loss f^3	Pr[block lost] (bound)
0.05	1.3×10^{-4}	$< 10^{-30}$
0.10	1.0×10^{-3}	$< 10^{-20}$
0.20	8.0×10^{-3}	$< 10^{-11}$
0.30	2.7×10^{-2}	$< 10^{-6}$

Availability analysis. Quantify recoverability under random node loss. If each node is independently unavailable with probability f and a shard has r holders, that shard is lost only if all its holders are down, w.p. f^r, so it is available w.p. $p = 1 - f^r$. A block of T shards is recoverable iff at least K shards survive:

$$\Pr[\text{recoverable}] \;=\; \sum_{i=K}^{T} \binom{T}{i} p^{\,i} (1-p)^{\,T-i}.$$

Table 4 evaluates this for the saturated shape (K16 M8, r=3). A shard is lost only when all three independently chosen holders are simultaneously down (f^3), and the code still tolerates M=8 such losses; the block-loss probability is therefore dominated by $\binom{T}{M+1}(f^3)^{M+1}$ and is negligible even at a severe f=0.3. This is the quantitative reason fragmentation does not trade away durability: replication (r) protects each shard, and parity (M) protects the block.

Correlated and colluding holders. The durability argument above assumes node failures are *independent*; the f^r term is exactly this assumption. It is the model's main soft spot. If the r holders of a shard are correlated (same operator, same data center, or a storage pool under one administrator), losing or withholding that shard costs far less than f^r suggests, and a colluding set that holds all r copies of enough shards can selectively withhold a block while appearing online. HRW placement spreads holders by identity, not by operator, so it does not by itself defeat an adversary who registers many identities behind one back end (the registration PoW raises but does not eliminate that cost). Two defenses are partial today and strengthen with the specified work: replication r and parity M raise the number of distinct holders that must collude, and the storage-weighted election (built) plus future bond slashing for missed proofs make holding-but-withholding economically self-defeating. A concrete hardening, which the HRW layer admits without protocol change, is *anti-affinity placement*: rank holders not over raw node identities but over (operator-or-ASN, identity) pairs declared at registration, so the top-r holders of a shard fall in distinct failure domains by construction; an operator controlling a fraction ϕ of declared domains then holds at most $\lceil r\phi \rceil$ of any shard's copies, and tolerating a fully-correlated domain of that size needs only $r > 1/(1-\phi)$ rather than trusting independence. This shifts the assumption from "failures are independent" to "no single domain exceeds ϕ," which is auditable. We implement identity-based HRW today and flag domain-aware placement as the concrete next mitigation, stating the correlation risk as a real limitation rather than a solved problem.

Sybil and long-range attacks. Mass identities cost bond + registration PoW per identity. Rewriting finalized history is impossible by construction (§8); a heavier-but-late branch below the finality depth is refused. Above the finality depth, the heaviest honest branch wins.

Table 5: Threat model summary. ✓ = mitigated in the prototype; ○ = partial.

Threat	Mechanism	Status
Block injection	signed header + per-shard hash	✓
Tampered body	Merkle/body-hash commitment	✓
Lying shard server	committed shard hash, skip+ban	✓
Data withholding	availability-first canonical rule	○
Sybil identities	bond + one-shot registration PoW	✓
Equivocation	slashing with on-chain evidence	✓
Long-range rewrite	depth-based finality (F deep)	✓
Cross-network replay	chain-id bound into every tx	✓
Eclipse / routing	peer diversity only	○

Leader grinding. Because leader election is HRW($prevHash, h, t$), an adversary might try to bias future elections by choosing the content of a block it produces so that the resulting hash favors its own future rank. Two factors limit this. First, the registration PoW seeds each identity's HRW key, so an attacker cannot cheaply mint identities whose keys win specific slots. Second, influencing *prevHash* requires actually producing the parent block (already a won election) and only shifts the *next* slot probabilistically, not deterministically; grinding over many candidate blocks costs hashing for a marginal bias. The storage-weighted election (§6) further binds odds to a quantity, proven storage, that an attacker cannot grind by reshaping a block: it must actually answer storage-proof challenges to gain weight.

Bond economics. The bond must exceed the expected gain from misbehavior for slashing to deter it. Registration requires a minimum bond, and the slash burns it while rewarding the reporter a fraction, so an equivocating validator faces a certain loss against an uncertain gain. Setting the minimum bond and slash fraction relative to block reward is a parameter-tuning question we leave to deployment; the mechanism, not the specific values, is the contribution.

Equivocation. Double-signing is slashable with on-chain evidence, making it economically irrational. **Eclipse/networking** attacks are out of scope for this paper and mitigated only partially (peer diversity); we note them as a limitation. Table 5 summarizes the model.

12 Implementation

ShadowLedger is implemented in Go as a set of small, independently tested packages: erasure coding (over an RS library), rendezvous placement, the Proof-of-Storage engine, the chain manager (fork-choice tree, reorg, finality, state), the uint64 VM and the SHL compiler, the registration-PoW and faucet-PoW primitives, and the peer-to-peer layer. The network is deployed as a live prototype with a bootstrap node on a public cloud host plus additional peers; discovery uses DNS seeds and peer exchange with no central coordinator. The deterministic core is exercised by an automated test suite spanning roughly sixteen packages. Table 6 lists the principal modules and their responsibilities; the separation keeps each

Table 6: Principal implementation modules and responsibilities.

module	responsibility
erasure	RS encode/decode, shard hashing, reconstruction
placement	HRW shard assignment, holder selection
consensus	Proof-of-Storage leader election, round fallback
chain	fork-choice tree, reorg, finality, block apply
state	accounts, validator registry, contract storage
vm + shl	uint64 VM, gas metering, SHL compiler
regpow	one-shot registration PoW (Sybil gate)
faucet	onboarding PoW, anchor verification
p2p	DNS-seed/peer-exchange discovery, gossip, sync

mechanism independently testable, which is how the security and functional checks of §13 are obtained in isolation before integration.

Networking and discovery. Membership is maintained without any central directory. A new node bootstraps from a small set of DNS seeds baked into the binary, then learns further peers by exchange (each peer shares a sample of its own peer set), so the view of membership grows and heals under churn, which is the same set the placement module feeds to HRW. Two channels carry traffic: a control channel gossips headers and full blocks for state application, and a shard-transfer channel serves and requests individual shards for reconstruction. Genesis is derived deterministically from the embedded parameters rather than gossiped, so every node that ships the same binary computes an identical block 0 and therefore an identical chain of prev-hash links, which is what lets a zero-configuration node join a running network safely.

Deployment notes. The prototype runs as zero-configuration mainnet: network parameters (genesis, seed nodes, erasure policy) are baked into the binary, so a freshly installed node joins automatically, in the spirit of how reference clients ship their genesis and seeds. The bootstrap node runs on a single small cloud instance behind a standard firewall; the only operational prerequisite is opening the control/transfer ports. Software updates are *advisory*: a node reports a newer release but never auto-applies it, since silent auto-update would be a central kill switch in a system whose point is decentralization. We treat the live network as a testnet: the tokens carry no real value, which lets us exercise consensus, reorg, finality, the faucet, and contract deployment on a genuinely public deployment without inviting financial risk before an external audit.

13 Evaluation

Methodology. We report *prototype* measurements, one cloud deployment plus a developer workstation, as evidence of feasibility, not statistical generalization (this is a Level-5 systems-design evaluation: an artifact exercised under controlled conditions). Erasure and placement experiments invoke the production encoder and HRW code paths directly rather than a re-implementation; storage figures average rendezvous placement of every shard over 2000 synthetic blocks per network size; throughput is the mean of 20 encode/decode repetitions per body size on a single core; PoW figures are medians over nine independently generated identities to capture the geometric-distribution variance of the search. Functional and security results come from the automated test suite and from exercising the live deployment.

Table 7: Per-node history storage vs. full replication (1 MiB blocks; rendezvous placement averaged over 2000 blocks). "max" shards/node stayed within 3% of the mean, confirming HRW balance.

n	spec	r	store/node/blk	reduction ρ
4	K3 M1	2	683 KiB	1.5×
8	K6 M2	3	512 KiB	2.0×
16	K11 M5	3	279 KiB	3.7×
32	K16 M8	3	144 KiB	7.1×
64	K16 M8	3	72 KiB	14.2×
128	K16 M8	3	36 KiB	28.4×
256	K16 M8	3	18 KiB	56.9×

Reproducibility. Every measurement here derives from a deterministic code path. The storage figures are a function of (n, K, M, r) and the rendezvous hash of fixed synthetic block identifiers, so they reproduce exactly given the same parameters; the throughput numbers depend only on body size and the single-core encoder; the PoW costs follow the geometric distribution of a fixed difficulty, which is why we report medians over independent identities rather than a single run. The functional and security checks are encoded as repeatable tests over the same deterministic core that runs in production, so a reviewer with the artifact can re-derive each result without access to our specific cloud host. We consider this the appropriate reproducibility bar for a single-team prototype, short of the controlled multi-system benchmark we leave to future work (§14).

13.1 Storage scarcity

We instantiate the model of §5.3 with 1 MiB block bodies, the network's adaptive (K, M) and r, and rendezvous placement of every shard over n nodes, averaged over 2000 blocks. Table 7 reports the bytes each node retains per block and the reduction versus full replication. The measured reduction tracks the analytical $\rho = Kn/(Tr)$: per-node history falls roughly as $O(1/n)$ while full replication stays at 1 MiB/block/node, reaching $\approx 28\times$ at n=128 and widening thereafter. Below n=4 fragmentation is not yet a win, as the model predicts.

13.2 Comparison with replication baselines

Storage alone is an incomplete metric; the right comparison holds *durability* fixed or reports both. Table 8 contrasts three schemes for a 128-node network and 1 MiB blocks: a classic full node (one whole replica per node), naive 3× whole-block replication distributed over the network, and ShadowLedger (K16 M8, r=3). Whole-block 3× replication is actually *cheaper* per node (24 KiB) than ShadowLedger (36 KiB), but it loses a block whenever the three holders of that block are all down ($\Pr = f^3 \approx 2.7 \times 10^{-2}$ at f=0.3), because there is no parity to recover from a partial set. ShadowLedger spends 50% more bytes to add erasure parity, buying roughly four orders of magnitude better durability ($\Pr[\text{loss}] < 10^{-6}$ at the same f, §11) while still storing $\approx 28\times$ less than a full node. The adaptive shape also keeps

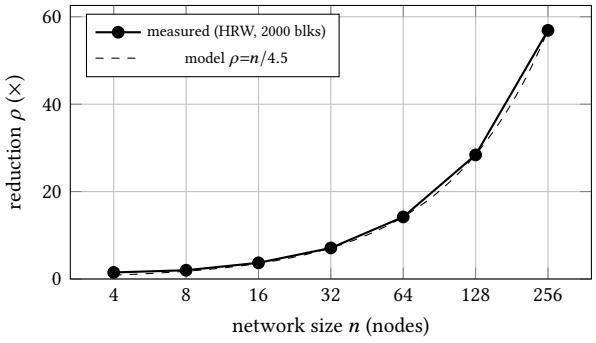

Figure 4: Per-node history-storage reduction vs. full replication. Measured rendezvous placement tracks the analytical $\rho = Kn/(Tr)$ once the erasure shape saturates ($n \gtrsim 32$).

Table 8: Storage vs. durability at n=128, 1 MiB blocks, f=0.3 node unavailability.

scheme	store/node	Pr[block lost]
full replica (classic node)	1024 KiB	≈ 0 (all hold all)
3× whole-block repl.	24 KiB	$\approx 2.7 \times 10^{-2}$
ShadowLedger (K16 M8,r3)	36 KiB	$< 10^{-6}$

Table 9: Faucet Proof-of-Work cost by difficulty N (leading zero bits).

N	median hashes	median time
12	3.1×10^3	3 ms
14	7.8×10^3	5 ms
16	1.5×10^5	65 ms
18	2.7×10^5	105 ms
20	5.7×10^5	267 ms

the data/parity ratio constant as n grows, so the durability margin does not erode with scale.

13.3 Coding cost and recoverability

At the large-network shape (K16 M8) the coding overhead is fixed at 1.5× (50% parity). RS encoding sustains 0.3–1.4 GB/s and reconstruction from exactly K shards 0.17–0.47 GB/s on one core for 64 KiB–4 MiB bodies, far above what a 5 s block interval requires, so coding is not a throughput bottleneck. We confirm the all-or-nothing threshold operationally: with exactly K shards the body is recovered and matches its commitment; with $K-1$ shards reconstruction fails (*not enough shards*) and yields nothing.

13.4 Onboarding cost

Table 9 reports faucet-claim cost by difficulty at $\approx 2.5 \times 10^6$ hashes/s on one core. Cost grows geometrically (+2 bits $\approx$ ×4), a tunable Sybil price; the deployed difficulty N=18 costs $\approx 2.6 \times 10^5$ hashes (~0.1 s), cheap for a genuine newcomer yet linear-in-identities for an attacker.

13.5 Parameter sensitivity

The behaviour above is governed by four knobs: data shards K, parity M, holders r, and finality depth F. Their effects are separable. Per-node storage scales as $S_{\text{node}} \propto (K+M)\, r/(K\, n)$, so it grows with the parity ratio M/K and with r, and shrinks with n; the reduction $\rho = Kn/(Tr)$ is what Table 7 traces. Durability is governed by M and r jointly: r sets how hard a single shard is to lose (f^r), and M sets how many shard losses the block survives, so the block-loss bound $\binom{T}{M+1}(f^r)^{M+1}$ falls steeply in both. Latency of audit/reconstruction depends on K (more data shards means more fetches to gather), and finality latency is exactly F block times. The policy's saturation at K=16, M=8, r=3 is the resulting compromise: a 50% parity ratio and triple holding keep the loss bound below 10^{-6} at f=0.3 (Table 4) while capping per-block shard count so coordination and per-shard overhead stay bounded as n grows; F is set so the mutable window (tens of seconds at a 5 s interval) is short for users yet ample for honest reconvergence. These are deployment choices, not fixed constants: a network that values durability over storage can raise M or r, paying the predictable storage cost above.

13.6 Cost of reconstruction (the tradeoff)

Fragmentation is not free: a full replica can read any historical body locally, whereas a ShadowLedger node must gather K shards, about $K \cdot (|b|/K) = |b|$ bytes of network transfer, to reconstruct one. Reading is thus a bandwidth/latency cost paid only on *audit, dispute, or sync*, not on normal validation (which touches state, §5). A late joiner syncing H historical blocks transfers $\Theta(H\,|b|)$ in the worst case of reconstructing every body, the same asymptotic volume a full replica downloads once; the difference is that the ShadowLedger node then *discards* all but its assigned shards, so its *steady-state* footprint is the $O(1/n)$ of Table 7 rather than the full archive. This storage-for-bandwidth trade is the expected shape for coded blockchains [15, 26]; ShadowLedger inherits it and confines the cost to infrequent operations.

13.7 Consensus behaviour

We exercised consensus in three regimes. With a single bootstrap validator the chain produces blocks at the target interval whenever there is work or a subsidy to mint, and idles otherwise, confirming the round-0 leader path. With a second validator joining via bonded registration, the HRW rotation alternates producers across heights without any reconfiguration round, since both derive the active set from identical on-chain state. Under an induced partition, two validators each extending their own tip, reconvergence occurs as soon as the heavier branch is observed: the lighter-tip node rewinds to its replay base and replays the winner, ending on the same head and state (verified by comparing post-reorg balances). Finally, holding a validator silent past one block time advances the round and lets a different validator produce, demonstrating the liveness fallback rather than a stall. These are qualitative confirmations on a small validator set, not a throughput benchmark; large-scale latency/throughput measurement is future work (§14).

Storage-weighted election. We verify the weighting is effective and deterministic: with two validators weighted 65:1 by proven storage, the high-storage validator wins > 90% of 2000 elections while the base-weight validator still wins occasionally, matching the

virtual-copy model of §6; every node computes the identical winner from the same chain state. On the public deployment we observe the loop end-to-end: the validator's auto-submitted storage proofs (§7) for protocol-assigned shards are accepted on-chain and its on-chain storage score rises (e.g. to several within the first minutes after the upgrade, visible in the validator registry), which the election weighting then reads. Block-production odds therefore track proven storage in the running system, not only in design or in unit tests.

13.8 Multi-node deployment

Beyond the analytical and single-node measurements, we ran a real *multi-process* cluster of *eight* independent node processes, each with its own data directory and ports, communicating over the actual P2P stack (DNS-seed plus peer-exchange discovery, block and shard gossip, snapshot sync) on a loopback network. This exercises real message passing, real cross-process shard transfer, and the real consensus engine, not a single-core simulation; it does not model wide-area latency or geographic partition, which we flag and leave to a distributed testbed (§14).

We observe the following on the cluster. (i) **Fast sync:** the seven follower processes joined from genesis and tracked the bootstrap node's chain through the real snapshot-plus-headers path, reaching a common tip with no manual intervention. (ii) **Permissionless registration:** four followers registered as validators (each locking a bond and solving the registration PoW), and the on-chain registry grew to five on every node with no reconfiguration round. (iii) **Storage proofs across nodes:** validators' auto-submitted storage proofs were accepted on-chain and their scores rose, visible to all nodes. (iv) **Storage-weighted rotation:** over a 30-block window all five validators produced blocks (8, 7, 7, 4, 4), the leader rotating across the set rather than concentrating, with the storage-weighting of §6 acting in a live multi-validator setting rather than only in a unit test. (v) **Convergence at scale:** across registrations and node restarts the eight processes stayed within one to two blocks of one another and re-converged to a single tip, with no persistent fork.

We note a methodological caveat learned here: on a single host, stale data directories and orphaned processes holding ports produce spurious "forks" that are experiment-hygiene artifacts, not protocol faults; a clean run converges. As defensive robustness we nonetheless added a guard so a node never extends a branch while a peer is ahead (it re-syncs instead), which removes a real failure mode for a validator that falls behind. Wide-area latency, partition, and large-n churn remain the right next experiment (§14); a loopback cluster shows the mechanisms compose and converge but cannot model the public Internet.

13.9 Functional and security checks

- **End-to-end.** The deployed network produced and validated several thousand blocks at a 5 s target interval; the automated test suite passes across ~16 packages.
- **Tamper resistance.** Seven forged-block variants (§11) are all rejected; an outsider cannot inject a block.
- **Convergence and finality.** Two validators partitioned and re-merged reconverge via rewind-and-replay; a reorg below F=16 is refused by construction.
- **Capital-free onboarding and contracts.** A zero-balance wallet earned its first tokens on the live network via the faucet; a fungible-token contract written in SHL was deployed and exercised (deploy, initialize, transfer, query) on the live network.

14 Discussion and Limitations

We report these candidly, per ACM/COPE integrity expectations. (1) **Storage coupling, built and remaining.** Leader election is now storage-weighted via on-chain storage-proof records (§6, 7): the running consensus is genuinely storage-coupled. Two related items remain. *Fork-choice* weight is still chain length (we weight *who leads*, not yet *which branch wins*), and missed storage proofs incur election-weight decay rather than a *bond* slash. Adding storage weight to fork choice and bond slashing for missed proofs are the priority next steps and would close the gap between the leader-level and branch-level coupling. (2) **State commitments**: the header commits transaction and log roots but not a *state root*, so light clients trust state via replay/snapshot rather than a succinct proof; adding a state (or verkle) root would enable trust-minimized light clients [4]. (3) **Data availability** is an operational reconstruction check, not a sampling proof; DAS [12] is complementary. (4) **Evaluation scale**: a single deployment and workstation, not a large multi-region benchmark; numbers are feasibility evidence. (5) **No external security audit**; the live network is treated as a testnet with no real value. (6) The contract VM is a Solidity subset. None of these undermines the central result, that history can be fragmented so no node stores it all, while state stays local and block production is earned by storage, but each bounds how far the present prototype should be trusted.

Applicability. ShadowLedger is a fit when history grows large relative to active state and many modest nodes are available to share it: public ledgers, audit logs, and data-preservation chains, where the archive dwarfs the working set and decentralization is valued over raw throughput. It is a poor fit when the working state itself is enormous (then keeping state local is the bottleneck, and state sharding as in OmniLedger [18] or RapidChain [34] is the right tool), or when historical reads are frequent and latency-critical (the reconstruction cost of §13 would dominate). The design also assumes enough independent nodes that the r holders of a shard fail independently; in a tiny or collusive network that assumption, and with it the availability argument of §11, weakens. We therefore present ShadowLedger as a point in the design space, not a universal replacement for replicated ledgers.

On head-to-head benchmarking. We deliberately do not report wall-clock comparisons against deployed systems such as Filecoin, Chia, or other coded-blockchain prototypes. Such numbers would be measured on different hardware, networks, and workloads, so a cross-system performance table would be apples-to-oranges and, by our own integrity standard, misleading; reproducing each system in a controlled common testbed is a substantial effort beyond this paper. Instead we compare where comparison is fair: analytically and under identical assumptions against full replication and naive distributed replication (§13), and qualitatively against prior coded-storage designs (§3). The distinction worth stating plainly is not a larger constant-factor saving than prior coded nodes, which

report substantial per-node reductions at a *fixed* network size; it is that ShadowLedger's reduction *scales* as $O(1/n)$ with the membership and is produced by the same mechanism that grants block-production rights, rather than by a storage layer bolted onto an unchanged consensus. A controlled multi-system benchmark on shared infrastructure is the natural next empirical step.

15 Design Rationale

Several choices warrant justification beyond their mechanics.

Why fragment history but not state? A naive "shard everything" design would force a network round-trip to read any balance during block validation, making validation latency nondeterministic and consensus fragile. By keeping the bounded, derivable *state* local and fragmenting only the unbounded *history*, validation stays a local, deterministic memory operation while the cost that actually grows without bound is the one distributed (§5). This mirrors how Bitcoin and Ethereum already keep a working set (UTXO set / state trie) locally, ShadowLedger simply removes the obligation to also keep the *archive* locally.

Why HRW rather than consistent hashing? Shard placement must rebalance gracefully as the permissionless validator set churns. HRW's per-key independent ranking moves only the shards a joining/leaving node would win/lose, whereas ring-based schemes can shift unrelated keys [16, 29]. Reusing the same primitive for leader election keeps the trusted-randomness surface small.

Why PoW only as a gate? Energy-as-security is the property we set out to remove. Yet a permissionless system still needs a per-identity cost (Sybil resistance) and an onboarding ramp for the capital-less. Confining PoW to a one-shot registration gate and a small faucet preserves those properties at negligible aggregate energy, while block ordering, the high-frequency operation, burns nothing. This is the Elastico insight [19] applied to a storage-centric, rather than throughput-centric, design.

Why depth-based finality first? A full finality gadget (e.g. Casper FFG [5]) adds voting rounds and messages. Depth-based finality reuses the reorg machinery already needed for fork choice: making the replay base monotone is sufficient to render deep history irreversible, with safety inherited from the common-prefix theorem [10]. It is the smallest change that yields hard irreversibility, and it composes with a future voting gadget.

16 Conclusion

ShadowLedger reframes the scarce resource of a blockchain from computation to storage. With a running prototype we show that a chain can fragment its *history* across the network, so no node stores it all, while keeping *state* local for fast validation, earning block-production rights by storage-weighted election instead of PoW, and remaining permissionless through bonded registration and a PoW faucet. An analytical model and measurements show per-node history cost falling as $O(1/n)$ ($\approx 28\times$ at 128 nodes) at a fixed $1.5\times$ coding overhead, alongside tamper rejection, partition reconvergence, hard finality, storage-weighted leader election verified against on-chain proofs, and capital-free onboarding on the live network. The remaining work, extending storage weight to fork choice, bond slashing for missed proofs, succinct state commitments, data-availability sampling, and external audit, is well scoped, and we believe the state-versus-history separation is a reusable principle for storage-scarce ledgers.

Future work. Four steps would move the prototype toward a system trustworthy for real value. First, completing the storage coupling: *storage-weighted fork choice* (the running system weights leader election by proven storage but not yet branch selection) and *bond slashing* for missed proofs (today the penalty is election-weight decay), recorded against the same on-chain storage proofs already in use (§7). Second, *succinct state commitments*: adding a state (or verkle) root to the header so light clients verify balances with a short proof instead of trusting a snapshot. Third, *data-availability sampling* layered on the committed shards, upgrading the operational reconstruction check into a probabilistic guarantee for non-storing clients. Fourth, a *controlled multi-system benchmark* on shared infrastructure and an *external security audit* before the network carries value. Each is well scoped, and none requires revisiting the core thesis; together they would raise the design from a demonstrated prototype to a deployable storage-scarce ledger.

References

[1] Mustafa Al-Bassam. 2019. LazyLedger: A Distributed Data Availability Ledger With Client-Side Smart Contracts. arXiv:1905.09274 [cs.CR]

[2] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. 2021. Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients. In *Financial Cryptography and Data Security (FC 2021) (LNCS, Vol. 12675)*. Springer, 279–298. doi:10.1007/978-3-662-64331-0_15

[3] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2018. The Latest Gossip on BFT Consensus. arXiv:1807.04938 [cs.DC]

[4] Benedikt Bünz, Lucianna Kiffer, Loi Luu, and Mahdi Zamani. 2020. FlyClient: Super-Light Clients for Cryptocurrencies. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 928–946. doi:10.1109/SP40000.2020.00049

[5] Vitalik Buterin and Virgil Griffith. 2017. Casper the Friendly Finality Gadget. arXiv:1710.09437 [cs.CR]

[6] Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang, and Yan X. Zhang. 2020. Combining GHOST and Casper. arXiv:2003.03052 [cs.CR]

[7] Bram Cohen and Krzysztof Pietrzak. 2019. The Chia Network Blockchain. Chia Network, Inc. (green paper). https://www.chia.net/wp-content/uploads/2022/07/ChiaGreenPaper.pdf.

[8] Stefan Dziembowski, Sebastian Faust, Vladimir Kolmogorov, and Krzysztof Pietrzak. 2015. Proofs of Space. In *Advances in Cryptology – CRYPTO 2015 (LNCS, Vol. 9216)*. Springer, 585–605. doi:10.1007/978-3-662-48000-7_29

[9] Ben Fisch, Joseph Bonneau, Nicola Greco, and Juan Benet. 2018. Scaling Proof-of-Replication for Filecoin Mining. Protocol Labs Research; IACR ePrint 2018/678. https://research.protocol.ai/publications/scaling-proof-of-replication-for-filecoin-mining/.

[10] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. 2015. The Bitcoin Backbone Protocol: Analysis and Applications. In *Advances in Cryptology – EUROCRYPT 2015 (LNCS, Vol. 9057)*. Springer, 281–310. doi:10.1007/978-3-662-46803-6_10

[11] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. 2024. FRIDA: Data Availability Sampling from FRI. In *Advances in Cryptology – CRYPTO 2024 (LNCS, Vol. 14929)*. Springer. doi:10.1007/978-3-031-68391-6_9

[12] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. 2025. Foundations of Data Availability Sampling. *IACR Communications in Cryptology* 1, 4 (2025). doi:10.62056/a09qudhdj

[13] Tao Huang, Yu Quan, and Xianwei Zhang. 2022. Downsampling and Transparent Coding for Blockchain. *IEEE Transactions on Network Science and Engineering* 9, 4 (2022). doi:10.1109/TNSE.2022.3155385

[14] Ari Juels and Burton S. Kaliski. 2007. Pors: Proofs of Retrievability for Large Files. In *Proc. 14th ACM Conf. on Computer and Communications Security (CCS '07)*. ACM, 584–597. doi:10.1145/1315245.1315317

[15] Swanand Kadhe, Jichan Chung, and Kannan Ramchandran. 2019. SeF: A Secure Fountain Architecture for Slashing Storage Costs in Blockchains. arXiv:1906.12140 [cs.IT]

[16] David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. 1997. Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. In *Proc. 29th Annual ACM Symposium on Theory of Computing (STOC '97)*. ACM, 654–663. doi:10.1145/258533.258660

[17] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. 2017. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *Advances in Cryptology – CRYPTO 2017 (LNCS, Vol. 10401)*. Springer, 357–388. doi:10.1007/978-3-319-63688-7_12

[18] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 583–598. doi:10.1109/SP.2018.000-5

[19] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. 2016. A Secure Sharding Protocol for Open Blockchains. In *Proc. 2016 ACM SIGSAC Conf. on Computer and Communications Security (CCS '16)*. ACM, 17–30. doi:10.1145/2976749.2978389

[20] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87 (LNCS, Vol. 293)*. Springer, 369–378. doi:10.1007/3-540-48184-2_32

[21] Andrew Miller, Ari Juels, Elaine Shi, Bryan Parno, and Jonathan Katz. 2014. Permacoin: Repurposing Bitcoin Work for Data Preservation. In *2014 IEEE Symposium on Security and Privacy (SP)*. IEEE. doi:10.1109/SP.2014.37

[22] Debarnab Mitra and Lara Dolecek. 2019. Patterned Erasure Correcting Codes for Low Storage-Overhead Blockchain Systems. In *2019 53rd Asilomar Conference on Signals, Systems, and Computers*. IEEE, 1734–1738. doi:10.1109/IEEECONF44664.2019.9048887

[23] Tal Moran and Ilan Orlov. 2019. Simple Proofs of Space-Time and Rational Proofs of Storage. In *Advances in Cryptology – CRYPTO 2019 (LNCS, Vol. 11692)*. Springer. doi:10.1007/978-3-030-26948-7_14

[24] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf Whitepaper.

[25] Sunoo Park, Albert Kwon, Georg Fuchsbauer, Peter Gaži, Joël Alwen, and Krzysztof Pietrzak. 2018. SpaceMint: A Cryptocurrency Based on Proofs of Space. In *Financial Cryptography and Data Security (FC 2018) (LNCS, Vol. 10957)*. Springer. doi:10.1007/978-3-662-58387-6_26

[26] Doriane Perard, Jérôme Lacan, Yann Bachy, and Jonathan Detchart. 2018. Erasure Code-Based Low Storage Blockchain Node. In *2018 IEEE Int. Conf. on Internet of Things (iThings) / GreenCom / CPSCom / SmartData*. IEEE, 1622–1627. doi:10.1109/Cybermatics_2018.2018.00271

[27] Irving S. Reed and Gustave Solomon. 1960. Polynomial Codes Over Certain Finite Fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304. doi:10.1137/0108018

[28] Yonatan Sompolinsky and Aviv Zohar. 2015. Secure High-Rate Transaction Processing in Bitcoin. In *Financial Cryptography and Data Security (FC 2015) (LNCS, Vol. 8975)*. Springer, 507–527. doi:10.1007/978-3-662-47854-7_32

[29] David G. Thaler and Chinya V. Ravishankar. 1998. Using Name-Based Mappings to Increase Hit Rates. *IEEE/ACM Transactions on Networking* 6, 1 (1998), 1–14. doi:10.1109/90.663936

[30] Jiaping Wang and Hao Wang. 2019. Monoxide: Scale Out Blockchains with Asynchronous Consensus Zones. In *16th USENIX Symp. on Networked Systems Design and Implementation (NSDI '19)*. USENIX Association, 95–112. https://www.usenix.org/conference/nsdi19/presentation/wang-jiaping

[31] Shawn Wilkinson, Tome Boshevski, Josh Brandoff, and Vitalik Buterin. 2016. Storj: A Decentralized Cloud Storage Network Framework. Storj Labs, Inc. (whitepaper). https://www.storj.io/whitepaper.

[32] Changlin Yang, Kwan-Wu Chin, Jiguang Wang, Xu Wang, Ying Liu, and Zibin Zheng. 2024. Scaling Blockchains with Error Correction Codes: A Survey on Coded Blockchains. *Comput. Surveys* 56, 6 (2024). doi:10.1145/3637224

[33] Mingchao Yu, Saeid Sahraei, Songze Li, Salman Avestimehr, Sreeram Kannan, and Pramod Viswanath. 2020. Coded Merkle Tree: Solving Data Availability Attacks in Blockchains. In *Financial Cryptography and Data Security (FC 2020) (LNCS, Vol. 12059)*. Springer, 114–134. doi:10.1007/978-3-030-51280-4_8

[34] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. RapidChain: Scaling Blockchain via Full Sharding. In *Proc. 2018 ACM SIGSAC Conf. on Computer and Communications Security (CCS '18)*. ACM, 931–948. doi:10.1145/3243734.3243853